SAVLG-CBM Branch Architectures

April 14, 2026

1 Purpose

This note describes the two branch architectures used in the current `SAVLG-CBM` implementation:

- **shared branch architecture**
- **dual branch architecture**

The key difference is whether the model uses one shared concept head for both global concept prediction and spatial grounding, or whether it uses separate global and spatial concept heads.

2 Common SAVLG Pipeline

For an image x, the visual backbone produces spatial feature tensors. Let

$$\mathbf{f}_5 \in \mathbb{R}^{C_5 \times H_5 \times W_5}$$

denote the final-stage feature map, and optionally let

$$\mathbf{f}_4 \in \mathbb{R}^{C_4 \times H_4 \times W_4}$$

denote an earlier feature map when a multiscale branch is used.

For K concepts, SAVLG produces:

- a global concept logit vector
$$\mathbf{c}_{\text{global}} \in \mathbb{R}^K,$$
- a spatial concept map tensor
$$\mathbf{M} \in \mathbb{R}^{K \times H \times W},$$
- a pooled spatial concept vector
$$\mathbf{c}_{\text{spatial}} \in \mathbb{R}^K,$$
- and the fused concept logits
$$\mathbf{c}_{\text{final}} = \mathbf{c}_{\text{global}} + \alpha\, \mathbf{c}_{\text{spatial}}.$$

In the current implementation, the image-level concept BCE is applied to $\mathbf{c}_{\text{final}}$, so the spatial branch participates directly in concept supervision through the residual fusion term.

3 Architecture 1: Shared Branch

3.1 Definition

In the shared architecture, a single concept layer produces the spatial concept maps:

$$\mathbf{M} = s(\mathbf{f}), \qquad \mathbf{M} \in \mathbb{R}^{K \times H \times W}.$$

There is no separate global concept head. Instead, the same output tensor is used for both roles:

- the spatial path keeps $\mathbf{M}$ as the localization map
- the global path pools the same tensor to produce concept logits

Concretely, if the concept layer does not expose a separate `forward_both(...)` method, the implementation reuses the same tensor for both outputs:

$$\mathbf{G} = \mathbf{M}, \qquad \mathbf{S} = \mathbf{M},$$

where $\mathbf{G}$ denotes the tensor consumed by the global pooling path and $\mathbf{S}$ denotes the tensor consumed by the spatial path. Then the global concept logits are obtained by pooling the same map tensor:

$$\mathbf{c}_{\text{global}} = \text{Pool}_{\text{global}}(\mathbf{M}).$$

3.2 Computation Graph

The shared architecture therefore has the form:

$$x \to \text{backbone} \to \mathbf{f} \to s(\cdot) \to \mathbf{M}$$

$$\mathbf{c}_{\text{global}} = \text{Pool}_{\text{global}}(\mathbf{M}), \qquad \mathbf{c}_{\text{spatial}} = \text{Pool}_{\text{spatial}}(\mathbf{M}).$$

Finally,

$$\mathbf{c}_{\text{final}} = \mathbf{c}_{\text{global}} + \alpha \mathbf{c}_{\text{spatial}}.$$

3.3 Interpretation

This architecture forces one concept tensor to satisfy two jobs at once:

- **semantic discrimination**: the pooled map must predict concept presence well
- **spatial faithfulness**: the map itself should align with the annotated concept region

The advantage is simplicity. The disadvantage is coupling: the same parameters must optimize global prediction and localization simultaneously, which can make the representation less specialized.

4 Architecture 2: Dual Branch

4.1 Definition

In the dual architecture, the model has two distinct heads:

- a **global head** for concept prediction

- a **spatial head** for concept localization

  The implementation exposes this explicitly through:

$$\texttt{forward_global(...)} \quad \text{and} \quad \texttt{forward_spatial(...)}.$$

The forward pass returns

$$\texttt{global_outputs}, \texttt{spatial_maps} = \texttt{forward_both(...)}.$$

4.2 Split-Stage Dual Variant

The basic dual variant keeps the global path on conv5 features and allows the spatial path to use a configurable stage:

$$\mathbf{c}_{\text{global}} = g(\mathbf{f}_5), \qquad \mathbf{M} = s(\mathbf{f}_{\text{spatial}}),$$

where

$$\mathbf{f}_{\text{spatial}} \in \{\mathbf{f}_5, \mathbf{f}_4, \ldots\}.$$

Thus, the global and spatial branches are separate even if they may read from the same or nearby backbone stages.

4.3 Multiscale conv4+conv5 Dual Variant

The most important dual specialization in this codebase is the `multiscale_conv45` branch.

Global path. The global branch stays on conv5:

$$\mathbf{c}_{\text{global}} = g(\mathbf{f}_5).$$

When the VLG-style global head is used, this is effectively:

$$\bar{\mathbf{f}}_5 = \text{GAP}(\mathbf{f}_5), \qquad \mathbf{c}_{\text{global}} = W_g \bar{\mathbf{f}}_5 + b_g.$$

Spatial path. The spatial branch first fuses conv4 and conv5 features:

$$\tilde{\mathbf{f}}_4 = W_4 * \mathbf{f}_4, \qquad \tilde{\mathbf{f}}_5 = \text{Up}(W_5 * \mathbf{f}_5),$$

$$\mathbf{f}_{\text{fused}} = \text{ReLU}(\tilde{\mathbf{f}}_4 + \tilde{\mathbf{f}}_5),$$

and then applies the spatial concept head:

$$\mathbf{M} = s(\mathbf{f}_{\text{fused}}).$$

So the dual multiscale model uses:

- a dedicated **global conv5 head**
- a dedicated **spatial multiscale conv4+conv5 head**

4.4 Computation Graph

The dual architecture has the form:

$$x \to \text{backbone} \to (\mathbf{f}_4, \mathbf{f}_5)$$

$$\mathbf{c}_{\text{global}} = g(\mathbf{f}_5), \qquad \mathbf{M} = s(\mathbf{f}_{\text{spatial}} \text{ or } \mathbf{f}_{\text{fused}}),$$
$$\mathbf{c}_{\text{spatial}} = \text{Pool}_{\text{spatial}}(\mathbf{M}), \qquad \mathbf{c}_{\text{final}} = \mathbf{c}_{\text{global}} + \alpha \mathbf{c}_{\text{spatial}}.$$

4.5 Interpretation

The dual architecture allows specialization:

- the global branch can remain close to a strong image-level concept predictor, often warm-started from VLG
- the spatial branch can focus on localization quality
- the two branches are combined only at the concept-logit level

This separation is especially important when:

- the user wants to freeze the global head
- the global branch should preserve VLG semantics
- the spatial branch should use a different stage or a multiscale fusion

5 Global and Spatial Pooling

Once the spatial map tensor $\mathbf{M}$ is produced, the model pools it to obtain spatial concept logits. In the residual path, the implementation uses log-sum-exp pooling:

$$c_{\text{spatial},k} = \tau \log \sum_u \exp(M_{k,u}/\tau) - \tau \log |U|.$$

This acts as a smooth max over locations.

For the global path:

- if the head already returns a vector, no extra pooling is needed
- if the head returns maps, those maps are pooled to produce $\mathbf{c}_{\text{global}}$

6 Main Conceptual Difference

The core distinction is:

Property	Shared	Dual
Global head	implicit, reused from spatial maps	explicit dedicated head
Spatial head	same tensor as global path	separate branch
Specialization	weak	strong
Supports VLG-style global warm-start cleanly	limited	yes
Supports multiscale spatial fusion with separate global path	awkward	yes

7 Key Differences in Practice

The architectural difference is not just a code-level refactor. It changes what the model is allowed to specialize.

7.1 Shared Branch

- One concept tensor must support both image-level concept prediction and localization.
- The pooled global concept logits are obtained from the same tensor that is later interpreted as the spatial explanation map.
- This is simpler, but it couples semantic discrimination and spatial faithfulness tightly.

7.2 Dual Branch

- The global branch has its own head and can stay close to a strong VLG concept predictor.
- The spatial branch has its own head and can use a different feature source or a multiscale fused feature map.
- This decouples classification quality from localization quality and makes freezing or warm-starting the global branch much cleaner.

7.3 Why this mattered empirically

In the recent `resnet18` reproduction work, the main recovery did not come from the outside penalty alone. The bigger gain came from restoring the older branch factorization:

- `branch_arch=dual`
- `spatial_branch_mode=multiscale_conv45`
- VLG-style global warm-start
- optional freezing of the global head

That combination recovered the stronger sparse results much more clearly than the newer shared-branch runs.

8 Observed Performance

Table 1 summarizes the most relevant comparison from the recent `resnet18` experiments.

Run	Branch style	Outside	Full acc	ACC@5	AVGACC
Recent weak `conv5` run	shared	0.0	0.7561	0.6929	0.7362
Old-style matched run	dual + multiscale_conv45	0.0	0.7591	0.7566	0.7574
Old-style matched run	dual + multiscale_conv45	0.25	0.7591	0.7577	0.7585

Table 1: Representative `resnet18 conv5` comparison. The main gap is not explained by outside penalty alone. Restoring the old dual-branch recipe recovers most of the lost sparse performance.

Interpretation. The full accuracies are all close, but the sparse accuracies are not. The shared-branch run stays around

$$\text{AVGACC} \approx 0.736,$$

whereas the old-style dual-branch reproductions recover

$$\text{AVGACC} \approx 0.758.$$

So the main practical difference is:

- **shared** is simpler but underperforms on sparse concept prediction
- **dual** preserves a stronger global concept predictor while allowing a specialized spatial branch

9 Why the Dual Architecture Performed Better

The recent reproduction work suggests that the stronger older results were not explained by outside penalty alone. The more important change was restoring the older *dual* branch recipe:

- separate global and spatial heads
- multiscale conv4+conv5 spatial fusion
- VLG-style global warm-start
- optional freezing of the global head

In other words, the main gain came from restoring a better branch factorization, while the outside penalty provided only a smaller additional improvement.

10 Summary

- The **shared** architecture uses one concept map tensor for both global and spatial roles.
- The **dual** architecture explicitly separates the global concept path from the spatial localization path.
- The multiscale conv4+conv5 model is a specialized **dual** architecture: global on conv5, spatial on fused conv4+conv5.
- The dual design better supports warm-starting, freezing, and branch specialization, which matches the empirical recovery of the stronger older `resnet18` results.

11 Backbone Result Tables

This appendix collects the current headline performance and localization numbers for the two backbones used most heavily in this project. The localization protocols are not identical across all rows, so each table explicitly states the metric family being used.

11.1 ResNet-18: Cross-Method Summary

Method	Full acc	ACC@5	AVGACC	mean IoU	mAP@0.3	mAP@0.5	mAP@0.7
LF-CBM	0.7402	0.5147	0.6817	0.0750	0.0366	0.0361	0.0364
SALF-CBM	0.7320	0.5335	0.6826	0.0789	0.0306	0.0295	0.0287
VLG-CBM	0.7594	0.7546	0.7556	0.0874	0.1054	0.0491	0.0288
SAVLG-CBM (best residual, $\alpha = 0.20$)	0.7606	0.7585	0.7585	0.1062	0.1174	0.0546	0.0313

Table 2: ResNet-18 cross-method summary. Localization numbers here are the matched post-hoc Grad-CAM comparison from `docs/cub_results.md`.

Method	mean IoU	mAP@0.3	mAP@0.5	mAP@0.7	point hit	coverage
LF-CBM (Grad-CAM)	0.0750	0.0366	0.0361	0.0364	0.5626	0.0334
SALF-CBM (Grad-CAM)	0.0789	0.0306	0.0295	0.0287	0.7284	0.0446
VLG-CBM (Grad-CAM)	0.0874	0.1054	0.0491	0.0288	0.8000	0.2536
SAVLG-CBM (Grad-CAM)	0.1062	0.1174	0.0546	0.0313	0.9011	0.1341
SALF-CBM (native)	0.0778	0.00645	0.00500	0.00456	–	–
SAVLG-CBM (native)	0.2820	0.4268	0.1790	0.0356	–	–

Table 3: `Resnet18` localization comparison. The first four rows are the strict matched `gt_present` mean-threshold Grad-CAM comparison across `LF`, `SALF`, `VLG`, and `SAVLG`. The final two rows append the best available native-map references for `SALF` and `SAVLG`, so they should be read as native reference rows rather than as part of the matched Grad-CAM protocol.

Method / run	mean IoU	mAP@0.3	mAP@0.5	mAP@0.7	point hit	coverage	Notes
SALF-CBM (native pairwise artifact)	0.0778	0.00645	0.00500	0.00456	–	–	native SALF vs SAVLG comparator artifact
SAVLG-CBM (native clean-global pairwise artifact)	0.0233	0.00620	0.00505	0.00458	–	–	paired with the SALF native artifact above
SAVLG-CBM (best residual `conv4+conv5`)	0.1309	0.1189	0.0656	0.0311	0.9889	0.3657	strongest earlier native `resnet18` row from the residual line
SAVLG-CBM (exact matched native rerun, `conv4+conv5`)	0.2820	0.4268	0.1790	0.0356	–	–	old native evaluator family, reproduced exactly

Table 4: `Resnet18` native-map localization references. `SALF` and `SAVLG` do not have a single fully matched native-map table covering all later `SAVLG` variants, so we separate the earlier paired `SALF`/clean-global-`SAVLG` artifact from the stronger later `SAVLG` residual native localizers. This table is useful for native-map context, but it is not a single strict four-method benchmark in the way Table 3 is.

SAVLG variant	Full acc	ACC@5	AVGACC	mean IoU	mAP@0.3	mAP@0.5	mAP@0.7
Soft-align + outside (warm-start frozen-global)	0.7591	0.7549	0.7560	0.1246	0.0491	0.0336	0.0237
Residual-Alpha-0.05	0.7592	0.7560	0.7568	0.1309	0.1189	0.0656	0.0311
Residual-Alpha-0.10	0.7585	0.7572	0.7581	0.1307	0.1189	0.0657	0.0310
Residual-Alpha-0.05 + local-MIL	0.7591	0.7563	0.7575	0.1216	0.0790	0.0434	0.0250
Residual conv5-only	0.7592	0.7560	0.7567	0.1251	0.1168	0.0630	0.0312
Residual conv4-only	0.7587	0.7547	0.7559	0.1445	0.1170	0.0710	0.0395
Old-style dual conv5, outside=0.25	0.7591	0.7577	0.7585	pending	pending	pending	pending
Old-style dual conv5, outside=0	0.7591	0.7566	0.7574	0.0543	0.0281	0.0034	0.00047

Table 5: ResNet-18 SAVLG internal ablations. Native localization numbers are reported where they are already available; the old-style outside=0.25 full-test localization rerun was interrupted before completion.

Method / run	Full acc	ACC@5	AVGACC	Loc. metric @0.3	Loc. metric @0.5	Notes
VLG-CBM (resnet50_cub_mm)	0.8529	0.8501	0.8533	0.1373	0.0528	Grad-CAM sweep
SALF-CBM (resnet50_cub_mm)	0.7263	0.2936	0.5540	0.0066	0.0051	Grad-CAM mean-threshold
SAVLG conv5 + cov025	0.8566	0.8558	0.8564	0.3301	0.1408	MaxBoxAcc sweep
SAVLG conv5 + cov050	0.8566	0.8544	0.8561	0.3207	0.1371	MaxBoxAcc sweep
SAVLG conv5 + dice010	0.8570	0.8558	0.8563	0.3393	0.1462	MaxBoxAcc sweep
SAVLG conv4	0.8584	0.8551	0.8559	0.3709	0.1573	MaxBoxAcc sweep
SAVLG conv4+conv5	0.8573	0.8565	0.8561	0.4467	0.2152	best localization
SAVLG conv5 soft-align only	0.8615	0.8570	0.8568	pending	pending	strongest conv5 classifier-side result

Table 6: ResNet-50 summary. SAVLG rows use native-map sweep metrics. VLG and SALF rows use the best available ResNet-50 Grad-CAM localization artifacts from the pod results directory, so this table mixes protocols and should be read as a broad comparison rather than a strictly matched localization benchmark.

11.2 ResNet-18: SAVLG Internal Ablations

11.3 ResNet-50: Backbone Summary

11.4 Compact Cross-Backbone Method Comparison

11.5 All Methods Across Both Backbones

11.6 Residual Pooling Ablation

Main takeaways.

- On `resnet18`, the main step change came from restoring the older dual-branch residual recipe, not from outside penalty alone.
- On `resnet18`, residual `conv4`-only gives the strongest native localization among the spatial-source ablations, while `conv5`-only and multiscale remain strongest on sparse accuracy.
- On `resnet50`, `conv4+conv5` is the clear localization winner, while `conv5 + soft-align only` is the strongest completed conv5 classifier-side run.

Run	mean IoU	mAP@0.3	mAP@0.5	Full acc	Notes
VLG-CBM (Grad-CAM, resnet50_cub_mm)	0.1091	0.1373	0.0528	0.8529	best available pod-side Grad-CAM sweep
SALF-CBM (Grad-CAM, resnet50_cub_mm)	0.0789	0.0066	0.0051	0.7263	best available pod-side Grad-CAM artifact
SAVLG conv5 + cov025	0.1824	0.3301	0.1408	0.8566	baseline conv5 branch
SAVLG conv5 + cov050	0.1793	0.3207	0.1371	0.8566	stronger coverage did not help
SAVLG conv5 + dice010	0.1854	0.3393	0.1462	0.8570	best finished conv5 localization
SAVLG conv4	0.2437	0.3709	0.1573	0.8584	best dense run in the 5-run sweep
SAVLG conv4+conv5	0.2822	0.4467	0.2152	0.8573	best localization in the branch sweep
SAVLG conv5 soft-align only	0.1241	pending	pending	0.8615	strongest completed conv5 classifier-side run
SAVLG soft-align + outside penalty	0.1246	–	–	0.7591	frozen-global warm-start local-loss ablation
SAVLG soft-align + local-MIL + outside	0.1255	–	–	0.7591	best reported ResNet-50 mean IoU

Table 7: Dedicated ResNet-50 SAVLG localization table. For the branch/loss sweep rows, mean IoU is the best value achieved over the explicit activation-threshold sweep, and the ‘mAP@0.3‘ / ‘mAP@0.5‘ columns correspond to the logged MaxBoxAcc sweep values at those IoU thresholds. The added VLG/SALF rows come from the best available pod-side Grad-CAM artifacts, so this table is useful for broad comparison but is not a perfectly matched single-protocol benchmark.

Backbone	Checkpoint	mean IoU	mAP@0.3	mAP@0.5	Full acc
`resnet18_cub`	SAVLG dual `conv4+conv5`	0.2820	0.4268	0.1790	0.7592
`resnet50_cub_mm`	SAVLG dual `conv4+conv5`	0.2822	0.4469	0.2152	0.8573

Table 8: Exact matched native-map comparison for the best SAVLG dual `conv4+conv5` checkpoint on `resnet18` and `resnet50`. Both rows use the same old native evaluator family, `map_source=native`, `map_normalization=concept_zscore_minmax`, fixed threshold sweep `0.05..0.95`, `concept_mode=intersection`, and `eval_subset_mode=gt_present`. This is the clean apples-to-apples localization comparison between the two backbones.

Backbone	Method	Full acc	ACC@5	AVGACC	Loc.@0.3	Loc.@0.5
`resnet18_cub`	LF-CBM	0.7402	0.5147	0.6817	0.0366	0.0361
`resnet18_cub`	SALF-CBM	0.7320	0.5335	0.6826	0.0306	0.0295
`resnet18_cub`	VLG-CBM	0.7594	0.7546	0.7556	0.1054	0.0491
`resnet18_cub`	SAVLG-CBM (best residual)	0.7606	0.7585	0.7585	0.1174	0.0546
`resnet50_cub_mm`	LF-CBM	N/A	N/A	N/A	N/A	N/A
`resnet50_cub_mm`	SALF-CBM	0.7263	0.2936	0.5540	0.0066	0.0051
`resnet50_cub_mm`	VLG-CBM	0.8529	0.8501	0.8533	0.1373	0.0528
`resnet50_cub_mm`	SAVLG-CBM (best localization)	0.8573	0.8565	0.8561	0.4467	0.2152

Table 9: Compact cross-backbone comparison using only the headline method rows. For `resnet18`, localization columns are the matched Grad-CAM comparison. For `resnet50`, SAVLG localization uses native-map MaxBoxAcc sweep results, while VLG and SALF use the best available pod-side Grad-CAM artifacts. This table intentionally omits auxiliary SAVLG ablations such as local-MIL and intermediate residual variants.

Backbone	Method	Full acc	ACC@5	AVGACC	Loc.@0.3	Loc.@0.5
`resnet18_cub`	LF-CBM	0.7402	0.5147	0.6817	0.0366	0.0361
`resnet18_cub`	SALF-CBM	0.7320	0.5335	0.6826	0.0306	0.0295
`resnet18_cub`	VLG-CBM	0.7594	0.7546	0.7556	0.1054	0.0491
`resnet18_cub`	SAVLG-CBM	0.7606	0.7585	0.7585	0.1174	0.0546
`resnet50_cub_mm`	LF-CBM	N/A	N/A	N/A	N/A	N/A
`resnet50_cub_mm`	SALF-CBM	0.7263	0.2936	0.5540	0.0066	0.0051
`resnet50_cub_mm`	VLG-CBM	0.8529	0.8501	0.8533	0.1373	0.0528
`resnet50_cub_mm`	SAVLG-CBM	0.8573	0.8565	0.8561	0.4467	0.2152

Table 10: All currently tracked headline method rows across the two main backbones. For `resnet18`, localization values are the matched cross-method Grad-CAM comparison. For `resnet50`, SAVLG localization values are native-map MaxBoxAcc sweep results, while VLG and SALF values come from the best available pod-side Grad-CAM artifacts.

Pooling	Full acc	ACC@5	AVGACC	Best mean IoU	Best Loc.@0.3	Best Loc.@0.5
`lse`	0.7591	0.7577	0.7585	pending	pending	pending
`avg`	0.7606	0.7561	0.7576	0.05435	0.02805	0.00331
`topk (0.2)`	0.7613	0.7566	0.7579	0.05435	0.02805	0.00331

Table 11: Residual pooling ablation for the old-style dual `conv5` `resnet18` recipe with outside penalty. `avg` and `topk(0.2)` native-map localization were evaluated with an explicit threshold sweep.